



Can Learned Indexes be Built Efficiently? A Deep Dive into Sampling Trade-offs

MINGUK CHOI, Dankook University, South Korea

SEEHWAN YOO*, Dankook University, South Korea

JONGMOO CHOI*, Dankook University, South Korea

By embedding the distribution of keys in indexing structure, learned indexes can minimize the index size and maximize the lookup performance. Yet, one of the problems in the present learned index is the long index-building time. The conventional learned index requires a complete traversal of the entire dataset, which makes it less practical than traditional index. This paper challenges the efficiency of build time to make the learned index practical.

Our approach for a build time-efficient learned index is to employ sampled learning. In this paper, we present two error-bounded sampling schemes: Sample EB-PLA, and Sample EB-Histogram. Although sampling is a simple idea, there are several considerations to make it practical. For example, sampling interval, error-boundedness, and index hyper-parameters are inter-related each other, presenting complicated trade-offs between build-time, index size, accuracy and lookup latency.

Throughout the extensive experiments over six real-world datasets, we show that the index-building time can be efficiently reduced over an order of magnitude by our sampling schemes. The results reveal that the sampling expands the design space of learned indexes, including the build-time as well as lookup performance and index size. Our Pareto analysis shows that a learned index can be built more efficiently than a traditional index through sampling.

CCS Concepts: • **Information systems** → **Data access methods**; • **Computing methodologies** → **Learning linear models**.

Additional Key Words and Phrases: Learned Index, Sampled Learning, Benchmark

ACM Reference Format:

Minguk Choi, Seehwan Yoo, and Jongmoo Choi. 2024. Can Learned Indexes be Built Efficiently? A Deep Dive into Sampling Trade-offs. *Proc. ACM Manag. Data* 2, 3 (SIGMOD), Article 116 (June 2024), 25 pages. <https://doi.org/10.1145/3654919>

1 INTRODUCTION

Employing machine learning techniques, learned indexes revolutionize the indexing schemes in large, sorted datasets [21]. This predictive mechanism significantly amplifies search efficiency in contrast to traditional structures like B-trees. Substantial validation has presented the prowess of learned indexes [12, 13, 49], fostering learned index to diverse applications such as dynamic

*Both authors contributed equally as corresponding authors.

Authors' addresses: Minguk Choi, mgchoi@dankook.ac.kr, Dankook University, Yongin, South Korea; Seehwan Yoo, seehwan.yoo@dankook.ac.kr, Dankook University, Yongin, South Korea; Jongmoo Choi, choijm@dankook.ac.kr, Dankook University, Yongin, South Korea.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2024/6-ART116
<https://doi.org/10.1145/3654919>

indexing [10, 16, 26, 30, 36, 41, 42, 45], multi-dimensional indexing [11, 28, 37, 46], sorting algorithms [22, 23], databases [7, 9, 24, 27, 31, 32, 43, 48], and embedded systems [40].

Despite the advantages, the learned index is hampered by the time-intensive nature of index building. Even in read-only environments, the learned index exhibits a longer construction time compared to traditional indexes. This limitation has been identified as a high-priority area for future work [8, 14, 18, 19, 21, 33, 34, 39]. Constructing a prediction model requires a complete dataset scan, which takes a long time, especially when the dataset is large. Although several approaches [6, 19, 27] are presented in the literature, sampling, one of the most crucial model-building techniques in machine learning, has not been carefully scrutinized in the learned index through sampling.

In learned index, sampling could make the model building be lightweight, reducing the model building time. This study delves deeply into the impact of sampling across a broad spectrum of performance metrics in learned indexes, highlighting the feasibility of sampling in learned indexes.

Introducing sampling poses a series of intricate challenges. The first and primary concern is to preserve error-bound properties. Several learned index implementations provide error-bound guarantees [14, 15, 19, 48], which compensate for inaccuracies in the prediction model. Incorporating sampling could further erode the model accuracy due to the reduction in data points used for learning, which could jeopardize the existing error-bound properties [29].

Second, the interplay of sampling parameters introduces intricate and intertwined relationships among a range of diverse performance metrics. These metrics encompass not only the index-building time but also index size, accuracy, and lookup latency. Successfully navigating these relationships hinges on a comprehensive understanding of these trade-offs and their implications, ensuring optimal outcomes when we employ sampling techniques.

Third, there is no benchmark that evaluates the performance impact of sampling. While some studies presented performance with sampling on traditional indexes [18, 33, 34], their assessments were influenced by different dataset structures and sample data access. That is, there is no common public benchmark for systematic evaluation and fair comparison among different approaches.

Addressing these challenges, this study introduces two sampling schemes for error-bounded learned indexes: **Sample EB-PLA** and **Sample EB-Histogram**. The proposed schemes refine the error bounds considering the sampling interval. The proposed schemes take a new input parameter for refined error bound. The refined error bound and the sampling interval are tunable parameters of indexes so that the user can flexibly control the performance according to the application requirements.

More than the error-bounded learned indexes, we explain how to apply sampling schemes on diverse indexes: learned indexes, histogram indexes, and tree-based indexes. Then, we thoroughly analyze the sampling impact on index-building time, index size, accuracy, lookup latency, and micro-architectural performances, encompassing a broad range of sampling parameters. This detailed study reveals the performance trade-offs and benefits tied to each approach, contributing comprehensive insights into their performance implications.

Thanks to our benchmark suites called **BASIL** (Benchmark of Sampling Applied Learned Indexes), we could experiment the sampled learning with various parameters, such as error bound and sampling interval. BASIL provides the common assumptions about sampled learning: common dataset structure and the sample data access. Thus, we can comparatively illustrate the sampling impact on diverse learned indexes in the wide design space.

Throughout extensive experiments, we evaluate the effectiveness of the sampling schemes on a variety of datasets. This validation assesses the adaptability and robustness, demonstrating that the observed benefits extend to various real-world data scenarios. By showcasing consistent positive outcomes across diverse datasets, the study reinforces the practical viability of the proposed

sampling techniques. The study also reveals that through sampling, the build time of learned indexes can be traded off with lookup performance and index size, effectively broadening the design space.

Based on the performance analysis, we introduce two noteworthy concepts: *Safe interval* and *Build speedup*. These metrics quantify the potential performance gains in the index-building time through down-sampling, with a marginal increase in lookup latency or index size. Additionally, this paper presents a *Pareto analysis*, elucidating performance trade-offs that govern construction time and lookup latency. By leveraging the power of sampling, this analysis reveals that a learned index can be more build efficient than a traditional index.

2 BACKGROUND AND LITERATURE REVIEW

This section provides preliminary information for learned indexes. We focus on read-only workloads for in-memory databases because it is the practical beginning point to incorporate the learned index. Still, there are applications that need to efficiently build an immutable index without the need for supporting individual updates. A prime example is the LSM-tree [2, 4, 5], where the build time of a read-only index is crucial for high performance in write-heavy workloads [9, 19, 31].

2.1 Index Definition and Terminologies

We define some terminologies regarding the index for clear definition. Let's consider a sorted array D of unique integers without duplicates, where each integer is denoted as k_i and i ranges from 0 to $|D| - 1$. Namely, we assume a duplicate-free dataset. For any arbitrary integer x in $\mathbb{Z}$, $Pos(x)$ returns the lower bound position in D . Precisely, if $Pos(x) = i$, then k_i denotes the smallest key in D that is greater than or equal to x . Upon being trained with pairs $(k, Pos(k))$ in D , an index constructs the model M , which offers a function $Pred(x)$ that predicts the lower bound position of x in D . Predicted position could be differ from the actual position, and the error in this case is defined as $Error(x) = |Pred(x) - Pos(x)|$.

Looking up a key, x , through the index involves two steps. The first step, *prediction*, estimates the position of x in D as p using $Pred(x)$. The second step is *correction*, where based on p , nearby keys are searched to find and return the exact position of x , denoted as $Pos(x)$. The *correction range* defines the maximum range where x can possibly exist in D and the size of this range is termed as the *correction length*.

Based on predefined error bound $\varepsilon (\geq 0)$, if $Error(k) \leq \varepsilon$ is guaranteed for all k in D , this is called the *error-bound* property [38, 47]. An index with such a characteristic is referred to as an *error-bounded* index. For the error-bounded index, the correction range is normally $[p - \varepsilon, p + \varepsilon]$ for all k in D , $[p - \varepsilon, p + \varepsilon + 1]$ for all x in $\mathbb{Z}$.

During the correction, various search methods can be utilized independently of the index. Error-bounded indexes typically use binary search within the correction range, but it may also employ linear or interpolation search [34]. Indexes that cannot provide a correction range generally perform an exponential search based on the predicted position p , occasionally using linear or interpolation search [10, 33].

2.2 Learned Indexes and Histogram Indexes

This section explains three representative learned indexes: recursive model index (RMI) [35], piece-wise geometric model index (PGM) [14], and radixspline (RS) [19]. We also include the compact hist-tree (CHT) [8], which is not a learned index, in our explanation. Because it searches a key based on a prediction model like learned index, rather than relying on traditional iterative comparison algorithms [8, 20, 39]. However, since CHT uses a frequency-based histogram model rather than a linear one, we categorize it separately as a histogram index. Four index structures are shown in Figure 1, and Table 1 summarizes the major features and characteristics of indexes. Kindly note that

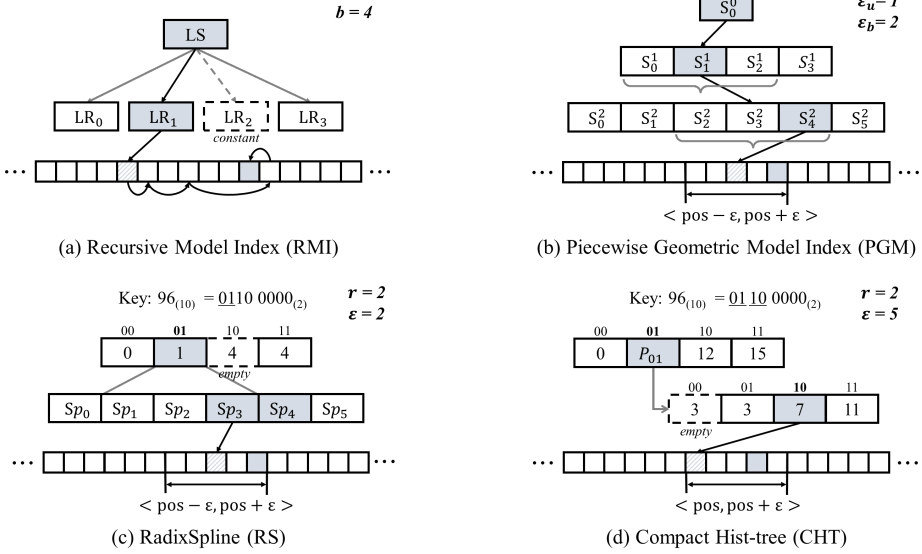


Fig. 1. Structure of indexes: RMI, PGM, RS, and CHT.

radix table (RT) [19] is also categorized as a histogram index, and we focus on the CHT explanation in this section.

2.2.1 Recursive Model Index (RMI).

Structure RMI is a multi-level learned index that supports various sub-models. While RMI can be configured in various ways [21, 33, 35], we focus on a two-level RMI: the top layer has a linear spline model, and the bottom layer has a linear regression model, as shown in Figure 1(a). It is known to be best suited when all keys fit in memory [33]. In the figure, the structure defines the branching factor b as the number of segments at the bottom layer for a single spline node in the top layer.

Building RMI trains the model from the top layer to the bottom layer. Traversing the dataset, the top layer's model constructs a linear spline that covers the entire key range using the first and the last key pairs. The top layer's model divides the key range into b number of fixed ranges (i.e., fixed segmentation). Each of these fixed ranges is assigned to one of the bottom layer's leaf segments, which are linear regression models. The bottom layer forms a piecewise linear regression (PLR) model through its segments, and each segment is trained to minimize the mean squared error (MSE) of the data within its range.

The number of keys present in each range can vary because RMI makes equidistant segments from the top layer's spline. Some segments could have not enough data points (zero or single data point) to conduct linear regression in the given key range. In this case, we create a *constant segment*, a segment with a constant value, as the dotted box in Figure 1(a).

The bottom layer model itself does not guarantee the error bound, although it minimizes MSE in each segment. Although there are studies to provide error bound for RMI [21, 33, 35], most of them incur significant overhead in index building. Thus, we focus on the RMI model without error bound.

Lookup During prediction, RMI first selects a bottom segment using the LS model in the top layer. The chosen bottom segment predicts the final position of the key in the dataset. During the correction, a search that does not require a correction range, such as exponential search, is

Table 1. Summary of the indexes.

Index	Model				Building		Correction		
	Top (Upper) Layer	Bottom Layer	# of Layers	Param.	Training Algorithm	Segmentation ¹	Construction Order	Error Bounded	Correction Range ²
RMI [21]	Linear Spline	PLR	2	b	Linear Least Squares	Fixed	Top-Down	X	$[0, D - 1]$
PGM [14]	EB-PLR	EB-PLR	≥ 2	ϵ_u, ϵ_b	Optimal PLR [47]	Dynamic	Bottom-Up	O	$[p-\epsilon, p+\epsilon]$
RS [19]	Histogram	EB-PLS	2	r, ϵ	GreedySplineCorridor [38]	Dynamic	Bottom-Up	O	$[p-\epsilon, p+\epsilon]$
CHT [8]	Histogram	EB-Histogram	≥ 1	r, ϵ	EB-Histogram [8]	Fixed & Dynamic	Top-Down	O	$[p, p+\epsilon]$

¹ Fixed divides the key range of segments equally, while dynamic divides it differently.² The p in correction range denotes the predicted position.

generally conducted based on the predicted position.

2.2.2 Piecewise Geometric Model Index (PGM).

Structure PGM is also a multi-layered learned index, as shown in Figure 1(b). In the figure, each layer has an error-bounded piecewise linear regression (EB-PLR) model. In contrast with RMI, PGM guarantees a predefined error bound (ϵ) in all predictions. We can configure the bottom layer's error bound (ϵ_b), separately from the upper layers' error bound (ϵ_u). While a dynamic variant of PGM exists which supports updates, the focus of this paper is on the standard, read-only variant of PGM [14].

Building PGM employs a bottom-up approach. Firstly, PGM traverses the dataset, and it constructs line segments in the bottom layer according to the EB-PLR. Note that EB-PLR gets the error bound from the user as a parameter of the algorithm. Traversing the dataset, EB-PLR finds the line segments that approximate the data points within a given error bound. If an additional data point is out of the given error bound from the current line segment, PGM completes the current one, and it creates an additional line segment on demand (i.e., dynamic segmentation).

Once the bottom layer has complete segments, the upper layer aggregates segments in the lower layer, preserving the error-bound property in its respective layer. The aggregation repeats until it reaches the top layer, which has a single segment.

Lookup PGM looks up the key from the top layer to the bottom layer, as shown in Figure 1(b). From the top layer, it predicts and finds the segment for the lower layer until it reaches the bottom layer. In the bottom layer, the position of the key in the dataset is predicted. Because EB-PLR provides an error-bound for prediction, we can search the key only within the correction range using binary search.

2.2.3 RadixSpline (RS).

Structure RS is a two-layered learned index, as shown in Figure 1(c). The bottom layer of RS constructs error-bounded piecewise linear splines (EB-PLS). Similarly to PGM, RS guarantees an error-bound(ϵ) for the prediction. The top layer of RS uses a radix table (RT) to index splines. We will further discuss the details of RT in the next CHT section.

Building RS is constructed bottom-up. In the bottom layer, RS employs EB-PLS that constructs the error-bounded piecewise linear splines by traversing the dataset. Similarly to PGM, the user can set the error bound as a parameter of the algorithm. RS dynamically segments splines, training data in the bottom layer. If the added data point is out of the given error bound, it completes the current spline, and it begins with a new spline. Then, the radix table in the top layer is updated accordingly.

Lookup For prediction, RS computes the lookup key's prefix value to directly access the corresponding segment (also called as bin in the RT terminology). It then probes the positions between the smallest spline in that bin and the subsequent bin, pinpointing two contiguous splines surrounding the lookup key. From these, the key's position is predicted through spline interpolation. Additionally note that RS also guarantees an error bound (ϵ), enabling a binary search within a

fixed correction range.

2.2.4 Compact Hist-tree (CHT).

Structure CHT is an error-bounded histogram index that constructs a tree-structured table, as shown in Figure 1(d). CHT uses RT as the histogram node, which is also used for the top layer in RS. RT uses the prefix radix bits (r) of a key as the index of the bin. RT divides the key range into a fixed number of 2^r bins with equal width (i.e., fixed segmentation). Each bin remembers the position of the smallest key in its range.

CHT calculates the frequency of each bin, which is the number of keys it belongs to, through the difference in the position of the smallest key between two consecutive bins. To ensure the error bound, CHT ensures that the frequency of all leaf bins is not more than the error bound (ϵ).

Building CHT is built in a top-down manner. CHT first constructs the root node in the top layer. When a bin's frequency exceeds the error bound, a child node is added to create a new histogram for its key range. This continues until every leaf bin's frequency is within the error bound in an on-demand manner (i.e., dynamic segmentation). As CHT divides key ranges into a fixed number of bins at the node level, some bins may have no data points according to the key distribution. Such bins are called *empty bins*, and each retains the position of the smallest trained key larger than its key range.

Lookup For prediction, CHT traverses through the tree until it reaches the leaf bin and returns the position of the smallest key in the leaf bin as a predicted position. Unlike RS or PGM, CHT permits only positive errors, resulting in the correction range of $[p, p+\epsilon]$ and the correction length of $\epsilon+1$.

2.3 Related Works: Sampling in Learned Indexes

There are a couple of related studies that tried to apply sampling to RMI. Flood [37] is a multi-dimensional index. It uses RMI to model the distribution of each dimension, and reduces the index-building time through sampling. Learned sorting [22, 23] makes a subset through sampling, and sorts the dataset using RMI. ALEX [10] is an updatable learned index. It uses linear regression like RMI, and reduces index-building time through sampling. They all rely on the linear regression model without an error bound, so they cannot guarantee the error bound.

Yaliang et al. [29] apply random sampling to learned indexes that use the error-bounded model, such as the FITing-Tree [15] and PGM [14], to reduce the index-building time. However, both indexes lost their error-bounded property due to sampling. As a result, they could no longer perform binary searches for correction, having a potential to suffer from increased correction overhead.

SOSD [18, 34] is the first benchmark that provides the comparison between traditional indexes and learned indexes. The proposed benchmark is popularly used in learned index studies to analyze the index sizes and the lookup performances. However, there are some limitations when comparing build times. The indexes are constructed from datasets of different data formats: key-only and key-value. Moreover, sampling is applied only to tree-based indexes, and even among these, there are differences in how they access the sample data. Some indexes use a newly created sample-only dataset, while others even touch unnecessary entries in the original dataset. We will explain this in detail in Section 4.2.

3 APPLY SAMPLING ON (LEARNED) INDEXES

This section presents the sampling schemes for various indexes. We apply the sampling to three learned indexes, two histogram indexes, and three tree-based traditional indexes. To clearly distinguish the sampled indexing, we give the prefix 's' to the name of the index (e.g., sRMI, sPGM, etc.).

Although sampled indexing is a simple and effective way to reduce the index-building time, we find two important considerations to make it practical when designing a sampled index. First, we can lose the error-bound property of the prediction model because sampling omits some training data points. Section 3.1 explains how we address this issue and how we actually apply sampling to diverse existing indexes.

Second, sampling changes the internal indexing structure according to the segment allocation schemes. For fixed segmentation indexes such as RMI, sampling could overly reduce the training data in the sparse key range. For dynamic segmentation indexes such as PGM and RS, sampling could increase the number of segments in error-bounded models. We explain these internal details in Section 3.2.

3.1 Sample Learning Methods Based on the Error Bound Property

Here, we explain how we can apply sampling to existing indexes. Using systematic sampling, we form S from D by extracting every I -th key from 0-th key, corresponding to the sampling interval (I), and the last key.

Recall that we assume datasets with unique keys, excluding duplicates. In a dataset with duplicates, the array index and lower bound position of a data point are not guaranteed to be identical. To determine the exact lower bound position of a sample key, non-sample keys should also be checked. This goes beyond our sampling method, so we leave it as future work.

Because sampling changes the error-bound property, we categorize the indexes into four: 1) error bound preserving model, 2) correction length preserving model, 3) correction length increasing model, 4) non-error-bounded model. We separately address the sampling schemes for each category.

3.1.1 Error-Bound Preserving Model: Sample EB-PLA for sPGM, sRS.

PGM and RS use EB-PLR and EB-PLS models, respectively. For brevity, we commonly refer to them as EB-PLA (Error-bounded Piecewise Linear Approximation). In Figure 2(a), EB-PLA used by PGM is trained with $\epsilon (=3)$ on all data and ensures an error bound for all data.

However, the original EB-PLA cannot preserve the given error bound when we apply sampling. Figure 2(b) shows the case of PGM with sampled learning. In the figure, the error bound is not guaranteed for some untrained triangular dots.

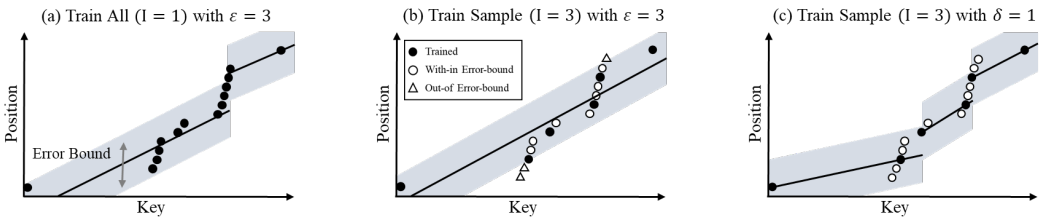


Fig. 2. Impact of sampling on error-bounded piecewise linear approximation.

We introduce **Sample EB-PLA**, a novel and general sample training method for the EB-PLA model [38, 47] used by PGM, RS, and others [9, 27, 32, 48]. The following Theorem 3.1 states that Sample EB-PLA scheme preserves the error-bounded property even when trained solely on samples. Sample EB-PLA trains all s in S with δ to preserve the error bound ϵ for all k in D , where I is the sampling interval and $\epsilon = \delta + I - 1$.

THEOREM 3.1. *Assume an algorithm constructs a EB-PLA model M with all s in S and $Pos(s)$ in D with the parameter $\delta (\geq 0)$. M is a monotonically increasing function that ensures $\forall s \in S, Error(s) \leq \delta$ in D . Then, M guarantees that $\forall k \in D, Error(x) \leq \epsilon (= \delta + I - 1)$ in D .*

It is recommended to set the δ as a multiple of I , as the frequency of each bin in Sample EB-Histogram normally increases in intervals. Figure 3(a) shows the CHT-trained model without sampling ($I=1$), with parameters $\epsilon=5$ and $r=2$. Figure 3(b) shows the sCHT-trained model with sampling ($I=3$) through Sample EB-Histogram with parameters $\delta=3$ and $r=2$. Figure 3(b) ensures the same correction length as Figure 3(a). Before sampling, the correction range is $[p, p+\epsilon]$ with only positive errors. After sampling, it shifts to $[p-I+1, p+\delta]$, including negative errors for keys at positions smaller than p .

3.1.3 Correction Length Increasing Model: sTree, sRT.

sTree is the tree-based index, which stores every I -th key and its position pair as a data node. During lookup, sTree finds the node with the smallest, but is not less than the lookup key. Then, sTree returns the position of the key, as p . Because the lookup key is larger than k_{p-I} and smaller than or equal to k_p , the correction range is $[p-I+1, p]$.

sRT constructs RT (radix table) with every I -th key. Each bin of sRT remembers the position of the smallest sample key in its range. If the position of the smallest sample key in the bin_n is p_n , keys in the range of $[p_n-I+1, p_n]$ can also be in the same bin_n . Thus, sRT's correction range is $[p_n-I+1, p_{n+1}]$. For both sTree and sRT, the correction length is increased by $I-1$ due to sampling.

3.1.4 Non-Error-Bounded Model: sRMI.

In RMI, the top layer can be trained with just the first and last data of the dataset, so there's no need to apply sampling. On the other hand, since the bottom layer trains all data, it can be built using sampling. In Figure 4(a), RMI's leaf segments minimize MSE for all keys in their range. In Figure 4(b) with sampling, sRMI only minimizes MSE for the sampled training keys within that range. Recall that RMI does not guarantee the error bound.

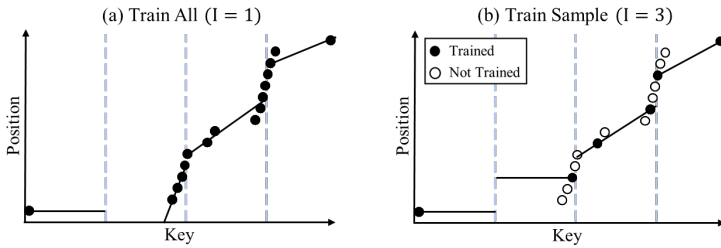


Fig. 4. Impact of sampling on RMI.

3.2 Internal Changes in Segmentation

Sampling affects not only the number of training keys but also the internal structure of indexes, which eventually affects the index-building time, index size, lookup performance. The impact of such sampling on the internal structure varies depending on whether the index dynamically divides the key ranges of sub-models or does so in a fixed manner.

Specifically, sPGM, sRS, and sTree dynamically segment key ranges according to the distribution. sRMI and sRT segment key ranges into a fixed number of segments or bins. sCHT utilizes both, segmenting key ranges into fixed-size bins while creating a new node dynamically.

3.2.1 Dynamic Segmentation Schemes.

The sampling interval should be carefully configured considering the error bound and dataset distribution. When Sample EB-PLA/Histogram learns from fewer data points (i.e., large sampling interval) with the same error bound (ϵ), the model might increase the internal segments, and consequently, the index size can grow.

Figure 2(a) and 2(c) comparatively present the model for the identical dataset, from EB-PLA and Sample EB-PLA, without and with sampling, respectively. In the figure, EB-PLA in (a) has two segments, but Sample EB-PLA in (c) has three segments. Although two models in (a) and (c) have the same error bound ($\epsilon=3$), Sample EB-PLA trains the model with $\delta=1$ to preserve the same error bound, and it leads to an increased number of segments in the model.

Figure 3(a) and 3(b) comparatively present EB-Histogram and Sample EB-Histogram, without and with sampling. In the figure, EB-Histogram without sampling has two histogram nodes and seven leaf segments; on the other hand, Sample EB-Histogram in Figure 3(b) has three histogram nodes and ten leaf segments. In the figure, the frequency of the fourth bin of root node is calculated to 4, which is larger than $\delta=3$; thereby, the model further splits the fourth bin.

Correction length increasing model, sTree, stores fewer keys as the sampling interval increases, resulting in a decrease in the number of data nodes. Consequently, the size of the index shrinks, and the key range managed by each data node broadens.

3.2.2 Fixed Segmentation Schemes.

As sRMI, sRT, and sCHT split the key ranges into a fixed number of segments regardless of the key distribution, some key ranges may not have enough data points for leaf segments to learn sufficiently, such as constant segments in RMI and empty bins in RT and CHT. Sampling increases the sparsity of the distribution by omitting data points, thereby increasing the number of insufficiently trained segments. The choice of the sampling interval should be thoughtfully adjusted based on the number of segments to split and the distribution of the dataset.

We can observe the changes of empty bins of CHT and constant segments of RMI in Figure 3 and 4, respectively. In Figure 3(a), without sampling, CHT has only one empty bin. In Figure 3(b), sCHT with sampling increases the number of empty bins from one to four. If an empty bin is used for lookup, the key exists in a position smaller than p , resulting in an inevitable negative error. In Figure 4(a), without sampling, the second line segment of RMI is not a constant segment. But in Figure 4(b), second segment of sRMI becomes constant due to sampling.

4 BENCHMARKING SETUP

We have applied sampling to several learned index schemes based on previous benchmark SOSD[18, 34]. This section presents the detailed prerequisites for benchmarking the indexes and required modifications for fair comparison.

4.1 Prerequisites for Indexes

We categorize indexing schemes into 4 categories.

Learned indexes: RMI, PGM, RS. We use RMI with improved build-time in [33]. We use PGM and RS from the reference papers [14, 19]. For a fair comparison of index-building time, we limit PGM to run with a single thread.

Histogram indexes: CHT, RT. We use CHT implementation by Stoian [39], and RT implementation by Kipf [19], respectively.

Tree-based indexes: ART, BTree, IBTree. STX BTree (BTree) [1], an interpolating BTree (IBTree) [17], and Adaptive Radix Trie (ART) [25] are used for tree-based indexes.

Baseline: Binary Search. For the baseline, we use the binary search provided by C++, which is `std::lower_bound` [3].

All indexes use the sampling interval (I) as a parameter to adjust the degree of sampling. Learned indexes and CHT use the parameters listed in Table 1. RT uses the radix bits (r) as a parameter. Tree indexes, similar to prior research [33, 34], do not use additional parameters beyond the sampling

interval. For the correction search algorithm, RMI uses exponential search based on the predicted position, while the other indexes use binary search based on the correction range.

4.2 Implementing (Learned) Indexes with Sampling

For fair comparison and considering the general usage scenario, we additionally confirm the following two implementation details in the existing indexes while preserving the original schemes.

First, we use a common data structure of dataset. RMI, PGM, and CHT were built with keys-only array datasets, which are half the size of key-value array datasets used by other indexes. We assume all indexes use key-value array during build.

Second, we use a common method to access the sampled entry in dataset. In previous benchmarks [18, 34], BTree was built with a newly created sample-only dataset. ART and IBTree unnecessarily accessed all the data in dataset array and determined whether it is a sample by checking its array index (position). We assume that indexes access only sample data from the entire dataset, incrementing array index by the sampling interval to access.

4.3 Datasets and Hardware

We use multiple datasets to quantitatively evaluate the performance impact of the proposed sampling on different indexing schemes. Because different datasets have different key-position distributions, the characteristics of the dataset affect the performance of indexes.

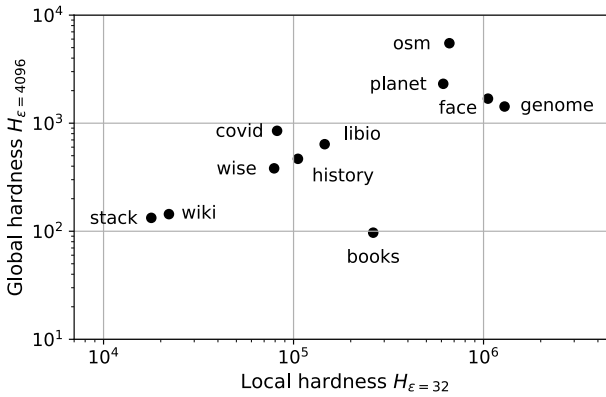


Fig. 5. Datasets used in the experiments.

Figure 5 presents the landscape of the eleven datasets, measured by a hardness index H_{ϵ} [44]. The global hardness in the Y-axis represents the linearity of the dataset with a large error bound ($\epsilon = 4,096$), and the local hardness in the X-axis represents the linearity of the dataset with a small error bound ($\epsilon = 32$). The error bounds of H_{ϵ} are chosen empirically in [44], which also aligns with the index performance observed in our experiments.

In this paper, we present the results from the 6 real-world datasets due to the page length limit: *books*, *osm* from SOSD [18] and *stack*, *covid*, *history* and *genome* from GRE [44]. Yet, similar results can be observed for nearby workloads. Each dataset contains 200 million key-value pairs, where the key is a 64-bit unsigned integer, and the value uses an 8-byte generated random payload. The *books* and *osm* datasets also have an additional *large* dataset with 800 million key-value pairs. We measure the lookup latency for uniformly chosen random 10 million keys from the dataset.

All experiments are conducted on a dual-socket machine with two 32-core Intel(R) Xeon(R) Gold 6338 CPU (2.00 GHz, 48 MB L3) and 512 GB of main memory.

5 EVALUATION

In the evaluation section, we specifically focus on the following questions.

- How does sampling impact the *performance*, *internal structure*, and *micro-architectures* of the each (learned) indexes? (§5.1)
- In variant real-world datasets, how much can we reduce the build time of an index through sampling without significantly compromising its size and lookup latency? (§5.2)
- How does the performance design space of the (learned) indexes change through sampling? (§5.3)
- Considering the average and tail latency, can learned indexes be built more efficiently than traditional indexes through sampling? (§5.4)

5.1 Performance Impact of Sampling

In this section, we present the performance of the indexing schemes (sRMI, sPGM, sRS, and sCHT) over different sampling intervals. We firstly focus on a representative dataset, *history*, because the local and global hardness of it is close to the median value. In the later section, we present more results with diverse datasets.

5.1.1 sRMI.

Figure 6 shows the performance result of sRMI along with diverse sampling intervals and number of segments. Specifically, we set the number of segments (b), ranging from 2^{10} to 2^{26} , and the sampling intervals, ranging from 2^0 to 2^{16} . The dataset has 200M keys, and sampling reduces the number of keys up to 3,052.

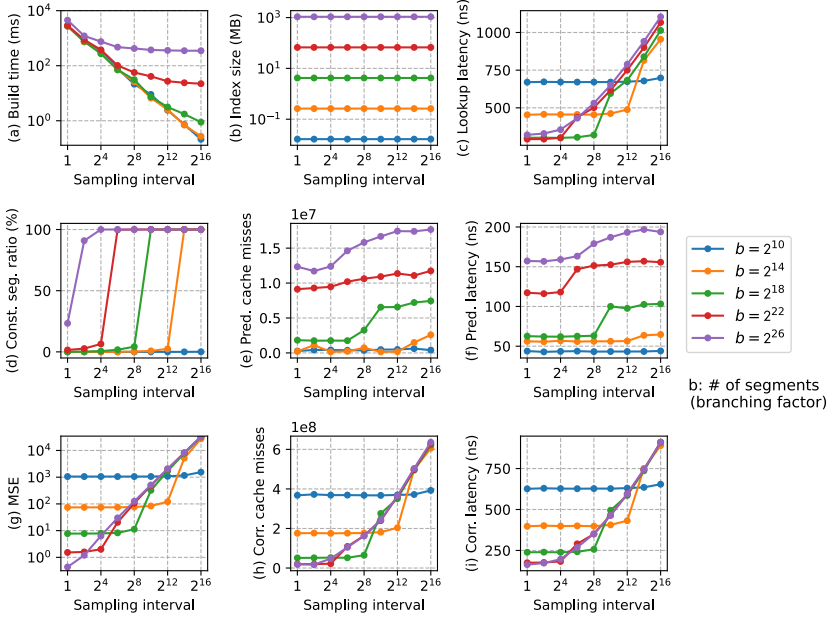


Fig. 6. Performance impact of sampling on sRMI on *history* dataset.

Build Time Figure 6(a) shows the index-building time according to the number of segments and sampling interval. As the sampling interval increases, the amount of training data decreases, resulting in a significant reduction in RMI's build time from 4.5 seconds to 211 microseconds. This clearly demonstrates that sampling can effectively reduce build time.

The index-building time decreases differently according to the number of line segments. For fewer segments ($b=2^{10}, 2^{14}$), sampling consistently and efficiently decreases the build time. For larger segments ($b=2^{22}, 2^{26}$), the reduction of build time starts to diminish at a certain interval. Each segment of the RMI must learn at least one key or, if not, be initialized to the last key position of the prior trained segment. Thus, regardless of the reduction in training data, build time converges to the time needed to simply initialize all segments.

Index Size, Cache, and Prediction Figure 6(b), 6(e), and 6(f) show the index size, the number of LLC (last-level cache) misses during prediction, and prediction latency, respectively. Figure 6(b) shows a consistent index size regardless of sampling interval, as the sRMI utilizes fixed memory based on the parameter b , the number of line segments. Prediction is the process where the CPU traverses through the index structure to get the prediction position. Hence, a small-sized index is expected to have a low prediction latency due to fewer cache misses. When all keys are learned ($I=1$), both Figure 6(e) and 6(f) show trends that align with our expectations. However, as the interval increases, we can observe the increased LLC misses during prediction as shown in 6(e) and increased prediction latency in 6(f), even without a change in index size. These can be attributed to other factors, such as model accuracy and constant segments, which we will explore further.

MSE, Cache, Constant Segment, and Correction Figure 6(g), 6(h), and 6(i) show MSE, the number of LLC misses during the correction, and the correction latency, respectively. When all data is learned ($I=1$), more segments lead to each segment covering a finer key range and points, resulting in a lower MSE in 6(g). A low MSE value means a small difference between the predicted position and the actual position. Thus, fewer entries are accessed during the correction, which in turn decreases LLC misses in 6(h) and correction latency in 6(i).

Figure 6(d) shows the proportion of constant segments according to the sampling interval. When all data is trained ($I=1$), except for when b is 2^{26} , the proportion of constant segments is near 0%. The proportion of constant segments remains close to 0% until a certain threshold interval and then suddenly spikes to 100%. As the number of segments increases, the threshold interval decreases. A reason is that large b creates numerous short key ranges with few data points. Thus, even with a small sampling interval, we can more frequently observe constant segments.

The sharp increase in constant segments significantly impacts the model's prediction accuracy. Until the threshold interval, MSE increases slightly because each segment is trained to minimize the MSE of the sample data, but not all. Once the threshold interval is reached, the increased constant segment returns the same position regardless of the key size, leading to a sharp rise in MSE in 6(g) and LLC misses during correction in 6(h) and correction latency in 6(i). Polluted cache during correction also affects prediction, leading to simultaneous increases in LLC misses during prediction in 6(e) and prediction latency in 6(f).

Table 2. Correlation coefficient between the performance counters with lookup latency.

Correlation	Instructions	Branch misses	TLB misses	LLC misses
Pred. Lat.	0.988	0.188	0.910	0.990
Corr. Lat.	0.984	0.995	0.928	0.997

Micro-architecture To understand the various micro-architectural effects of sampling, we measured various performance counters, including LLC misses, committed instructions, branch prediction misses, and TLB misses, during prediction and correction. Table 2 shows the correlation coefficient between the measured performance counters and the prediction and correction latency. All metrics correlate strongly with prediction and correction latency, except for branch misses with prediction latency. We focus on the LLC misses due to its highest correlation to explain both latencies.

Lookup Latency Finally, Figure 6(c) represents the lookup latency of sRMI based on the interval, which is the sum of the prediction latency in 6(f) and the correction latency in 6(i). When trained on all data, the lookup latency generally decreases with an increasing number of segments, except at $b=2^{26}$. When applying sampling, sRMI can maintain its lookup performance while reducing the build time until it encounters the threshold interval.

5.1.2 sPGM and sRS.

Figure 7 and 8 show the performance results of sPGM and sRS along with the sampling intervals, respectively. For both sPGM and sRS index, the bottom layer's error bound (ϵ) is configured from 2^2 to 2^{16} . Sampling is applied to the bottom layer, and the sampling interval is chosen from 1 to the respective ϵ , up to 2^{16} . The upper error bound for PGM was set to the default value of 4, and the radix bit for RS was set to the default value of 18.

Figure 7 and 8 show very similar trends because both indexes employ the EB-PLA model at the bottom layer. For the rest of the section, we first provide a common explanation for both sPGM and sRS. Then, we present the analysis by unique features.

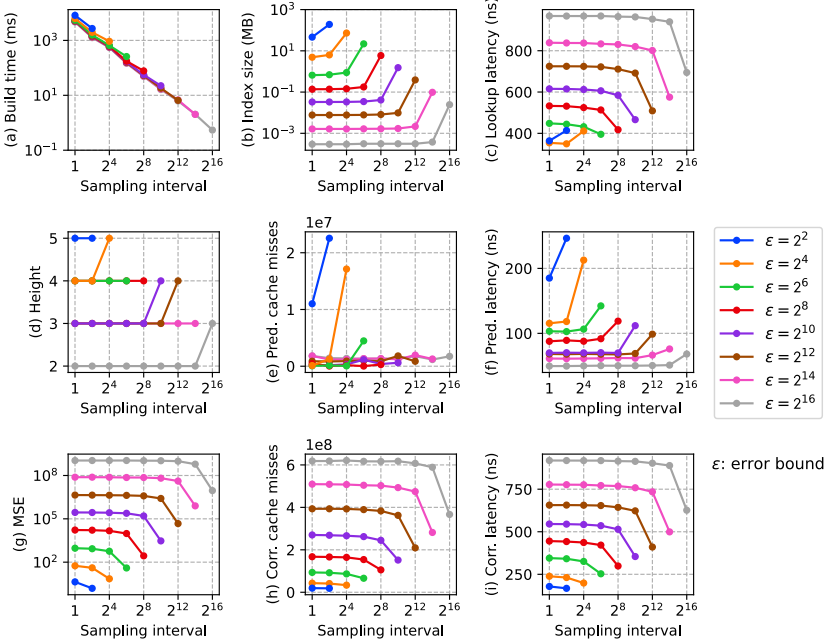


Fig. 7. Performance impact of sampling on sPGM on *history* dataset.

Build Time In both figures, (a) presents the index-building time. Without sampling, both indexes have long build times in few seconds. As the sampling interval increases, reducing the amount of data trained on, the build time drops from 8.2 seconds to 543 microseconds for sPGM and from 2.2 seconds to 318 microseconds for sRS. This underscores the significant performance improvement in build time, thanks to sampling. Note that due to epsilon constraints, an index with a small ϵ cannot be trained with larger intervals than $\epsilon + 1$.

Index Size, Height, Cache, and Prediction In Figure 7 and 8, (b) presents the index size. Figure 7(d) shows the index height of sPGM, and Figure 8(d) shows the average number of splines per bin for RS. In the figure, without sampling ($I=1$), a smaller error bound results in a larger index size. EB-PLA in the bottom internally forms more segments to ensure smaller ϵ , which is more challenging. The sPGM's upper layer, being also an EB-PLA model, grows in size and height with a

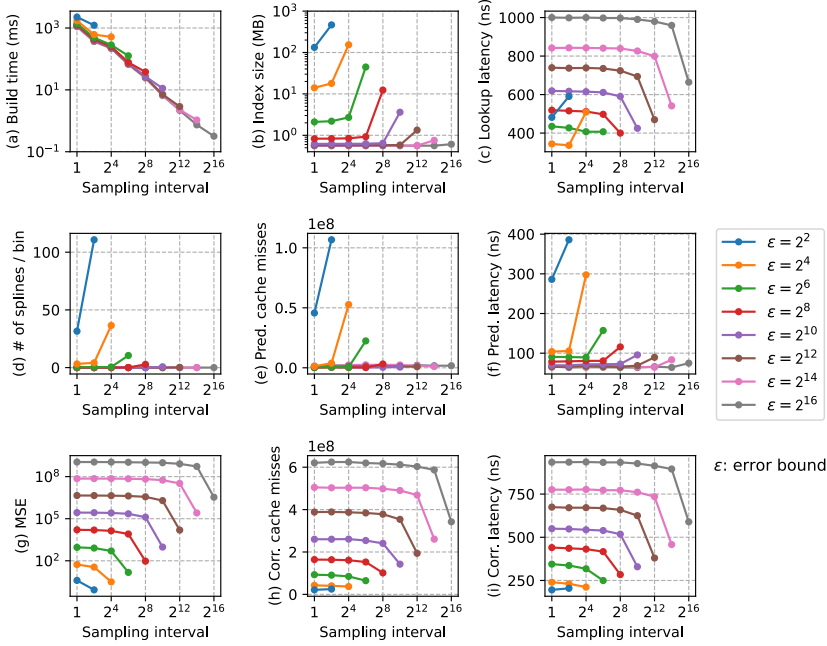


Fig. 8. Performance impact of sampling on sRS on *history* dataset.

smaller ϵ in Figure 7(d). sRS's top layer has a fixed-size RT, so a smaller ϵ results in an increased number of splines per bin in Figure 8(d). Due to the top layer's differences, sPGM's index size varies with the error bound, while sRS's difference is minimal when the error bound exceeds 2^{10} .

In both figures, the index size in (b) remains steady or slightly grows as the sampling interval increases, then sharply rises at a specific threshold interval. This is attributed to the EB-PLA needing more segments to ensure the same ϵ even when trained on fewer data. As the number of bottom segments increases, the size of the top layer in sPGM grows, and the average number of splines per bin in sRS also increases in Figure 8(d). Notably, when the sampling interval matches ϵ , the difficulty surges dramatically, leading to a significant increase in index size. In such cases, even the height of sPGM can increase in Figure 7(d).

In both figures, (e) and (f) show the cache miss during the prediction and prediction latency, respectively. Similarly to sRMI, prediction closely relates to the index size and LLC cache misses, as it touches through the index in memory. As the error bound decreases, the index size, height, and number of splines per bin increase, leading to an increase in cache misses in (e) and an increase in prediction latency in (f). Interestingly, sPGM, with its compact index size, shows reduced cache misses up to 2^{10} as the error bound increases, then slightly increases or stabilizes in cache misses beyond that due to correction cache misses.

For both sPGM and sRS, when the error bound is above 2^6 , the index size is small enough to fit within the cache boundary, resulting in minimal prediction misses in (e) and low prediction latency in (f). As the sampling interval grows, increases in index size, height, or splines per bin lead to more LLC cache misses in (e) and elevated prediction latency in (f).

MSE, Cache, and Correction In both figures, (g), (h), and (i) show MSE, the cache miss, and correction latency, respectively. Without sampling ($I=1$), the smaller the error bound, the smaller the MSE in (g). This is quite obvious since as the allowed maximum error decreases, the average error also diminishes. Both sPGM and sRS use binary search for the correction from the prediction position. When the error bound is small, the correction range is also small. Thus, a smaller error bound leads to fewer cache misses in (h) and lower correction latency in (i).

In (g), as the interval increases, MSE slightly decreases, which is favorable compared to sRMI. Due to sampling, the increased segments by Sample EB-PLA result in smaller key ranges and fewer data points per segment, leading to more accurate predictions. The decrease in MSE with the interval also reduces cache misses and latency during the correction in (h) and (i), respectively.

Lookup Latency Finally, the average lookup latency is presented in (c), which is the sum of the prediction latency in (f) and the correction latency in (i). As the interval increases, prediction latency rises while correction latency decreases. Based on the ratio of the two latencies as the interval grows, latency increases for $\varepsilon=2^2$, decreases then rises for $\varepsilon=2^4$, and decreases for the rest.

5.1.3 sCHT.

Figure 9 shows the performance of sCHT according to the sampling interval and the error bound. We set ε from 2^3-1 to $2^{17}-1$ to ensure that δ is a multiple of the I and to set a correction length similar to sPGM and sRS. The sampling interval is chosen from 1 to the respective $(\varepsilon+1)/2$, up to 2^{16} . sCHT does not have a default value for the radix bit(r), and we use ten bits ($r=10$), which can cover various error bounds in most cases.

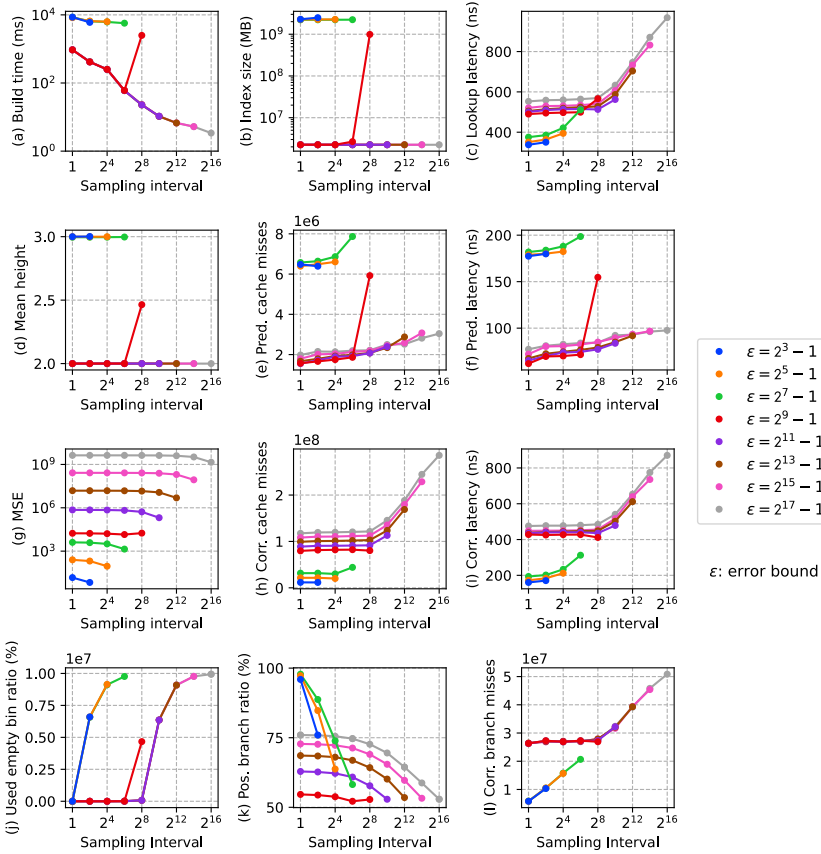


Fig. 9. Performance impact of sampling on sCHT on *history* dataset.

Build Time and Height Figure 9(a) and 9(d) show the index-building time and mean height. In Figure 9(d), without sampling ($I=1$), sCHT heights are grouped by $\varepsilon=2^7-1$: those below the threshold at height 2, and those equal to or exceeding it at height 3. While most learned indexes show little build time variance across configurations, sCHT has a significant difference between

groups. This is because sCHT doesn't use a single pass for index building. Instead, it iteratively accesses data in the key range of each added node for training. As a result, groups with a height of 3 access the dataset more than those with a height of 2 to train more nodes.

In Figure 9(a), we can observe that the build time decreases from 8.5 seconds to 5.7 seconds for groups with heights of 3, and from 0.9 seconds to 3 milliseconds for groups with heights of 2 as the sampling interval increases. As the sampling interval increases, the build time generally decreases. The decrease is more pronounced at a height of 2 compared to a height of 3. This highlights the notable enhancement in build time achieved through sampling. For sCHT with $\epsilon=2^9-1$ and $I=2^8$, it's a special case where the build time is longer due to the increased height. We will delve into the details later on.

Index Size, Height, Cache, and Prediction Figure 9(b) shows the index size. When trained on all data, the size of the index is remarkably similar if the height is the same. The size difference between the largest size with the smallest ϵ and the smallest size with the largest ϵ is 3% at height 3 and 0.2% at height 2. Unlike sPGM and sRS, the index size of sCHT doesn't increase gradually as ϵ decreases but rather increases step-wise based on specific threshold ϵ : the threshold ϵ lies between 2^7-1 and 2^9-1 in this case. Because sCHT fits the key distribution using a equal-width constant model without a slope, rather than using LR model.

As interval increases, in most cases, additional segmentation due to sampling rarely occurs. The height remains unchanged in 9(d), and the index size either remains consistent or slightly increases in 9(b). However, when aggressive sampling is applied near a threshold error bound like $\epsilon = 2^9 - 1$, height in 9(d), size in 9(b), and build time in 9(a) all surge.

Figure 9(e) and 9(f) show the cache miss and latency during the prediction. Without sampling, cache miss and latency in predictions are divided into two groups, with lower heights resulting in lower values. Within a group, smaller ϵ leads to slightly lower cache misses and latencies, even though the index sizes are nearly identical. Also, as the sampling interval increases, cache misses and latency slightly increase in both groups even though there are no significant changes in size. Both of these trends are due to the correction, which will be explained in the following section.

MSE, Cache, Branch, Empty Bin, and Correction Figure 9(h) and 9(i) show the cache misses and latency during correction. During the correction, there's a distinct gap in cache miss and latency between groups, while the difference within groups is subtle. This inter-group gap arises from the substantial size difference between groups, as seen in 9(b). Within a group, as the error bound decreases, the correction range becomes smaller, leading to slightly lower cache misses and latency. This also affects cache miss in 9(e) and latency in 9(f) during prediction. Nonetheless, the intra-group gap remains marginal because the differences in correction range have a limited impact, unlike sPGM and sRS. Thus, we further investigate another micro-architectural impact, observing branch prediction-related performance counters.

Figure 9(k) and 9(l) show the ratio of positive branch and branch prediction misses during correction. Despite varying error bounds within a group, index sizes remain consistent in 9(b). This implies that indexes within the group mostly share the same nodes and bins, and the position of the smallest key each leaf bin points to, p , is also the same; only the end of the correction range, $[p, p+\epsilon]$, varies with ϵ . Yet, the actual effective correction range where the lookup key exists is based on the smallest error bound within the group. For an error bound of $2^{17}-1$, it binary searches a range about 256 times broader than for 2^9-1 . However, within the binary search's while loop, the first 8 branches will be the positive branch where the mid is greater than the lookup key. Thus, within a group, a larger error bound increases the positive branch ratio, leading to biased branching in 9(k). Consequently, the branch prediction miss converges to the smallest error bound within the group in 9(l), making negligible differences in cache misses in 9(h) and latency in 9(i) during corrections within the group.

Figure 9(g) and 9(j) show MSE and the ratio of used empty bins during prediction. We measure MSE of sCHT based on the first middle position of binary search. Without sampling, a smaller ϵ results in a smaller MSE in 9(g), just like sPGM and sRS. As the interval increases, yet almost no bins are added as the index size remains consistent in 9(b), MSE decreases. This is because sCHT allows negative errors when we apply sampling, and the size of these negative errors increases with the interval. As shown in (k), the branch direction during the correction becomes complicated, including the negative region as well as the positive region. This results in higher branch prediction misses in 9(l). Moreover, even if the MSE decreases as the interval expands, as shown in 9(g), the increased branch misses in 9(l) lead to a high in cache misses in 9(h) and latency in 9(i). Particularly, as the interval expands and the proportion of empty bins that invariably cause a negative error during lookups increases in 9(j), this tendency becomes even more pronounced. Additionally, increased cache misses during correction affect the cache used also during prediction, leading to a slight increase in cache misses in 9(e) and latency in 9(f) during prediction.

Lookup Latency Finally, the average lookup latency is presented in Figure 9(c), which is the sum of the prediction latency in 9(f) and the correction latency in 9(i). As the interval increases, both prediction and correction time increase, leading to higher lookup latency.

5.2 Build Speedup by Sampling

This section presents possible performance gains from sampling. As we have observed in the previous sections, sampling can reduce the index-building time in most cases. Yet, the reduced build time comes at the cost of the index sizes or lookup performance.

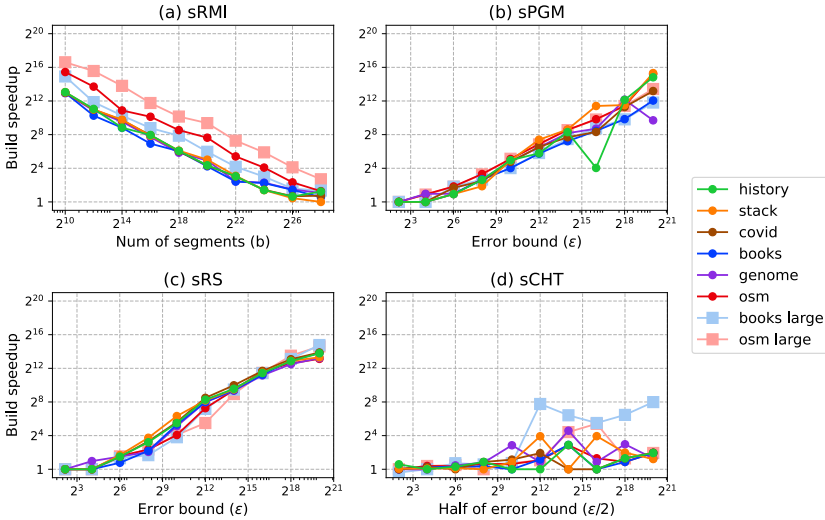


Fig. 10. *build speedup with safe interval.*

We investigate the possible speedup of the index-building time with minimal cost of another performance measure. Specifically, we define a *safe interval* as the maximum sampling interval, ranging from 1 to 2^{20} , under the condition that allows 5% of additional index size or lookup latency with respect to the baseline (non-sampling) performance. Then, we measured the improved index-building time (or *build speedup*) with respect to the baseline performance. Then, we measured the improved index-building time (or *build speedup*) with respect to the baseline performance.

Figure 10 shows *build speedup* across datasets based on index configurations. The x-axis denotes configurations: the number of segments for sRMI, error bounds for sPGM and sRS, and half of the

error bound for sCHT. sCHT's x-axis is adjusted to scale correction length with sPGM and sRS. Other parameters align with section 5.1. We use six datasets in Section 4.3, and two large datasets from *books* and *osm*.

In Figure 10(a), sRMI shows that with a smaller number of segments, even with a larger interval, lookup latency doesn't degrade significantly, resulting in a larger *safe interval* and substantial *build speedup*. In Figure 10(b) and 10(c), sPGM and sRS demonstrate that with a larger error bound, a larger interval doesn't significantly degrade the lookup latency or the index size; thus, it leads to a larger *safe interval* and significant *build speedup*. All three indexes exhibit some variance based on datasets, but they generally follow the same trends.

The results demonstrate that sampling can effectively improve the build time of the learned index. In a dataset with 200 million keys, with just an additional 5% in index size and lookup latency cost, the index building time can be effectively reduced to as much as 1/44,514 for sRMI, 1/40,781 for sPGM, and 1/14,479 for sRS.

In Figure 10(d), sCHT exhibits a significantly smaller and non-linear *build speedup* compared to other indexes. This is because as the sampling interval increases, unlike other indexes, sCHT increases the index size, prediction latency, and correction latency. Moreover, the interval in which latency does not sharply increase is relatively small. As a result, the *safe interval* of sCHT, that allows a 5% performance drop, is very narrow, and the *build speedup* is small. Specifically, until half of the error-bound reaches 2^8 , most of the *build speedup* is close to 1.

For large datasets, as the size of the dataset increases fourfold, the *safe interval* and *build speedup* of sRMI also increases by a factor of four in Figure 10(a). This is because the index size of sRMI is determined solely by the number of segments, irrespective of the dataset size. sPGM and sRS present similar *safe interval* and *build speedup* in large datasets as in smaller datasets, since the size of the EB-PLA model changes as the dataset increases. In Figure 10(d), sCHT achieves higher *safe interval* and *build speedup* in the large dataset, particularly at larger error bounds. Because the large dataset has more data points which can mitigate the growth of empty bins. In summary, as the dataset size increases, the *safe interval* and *build speedup* increase proportionally in sRMI, remain similar in sPGM and sRS, and increase in sCHT when the error bound is large.

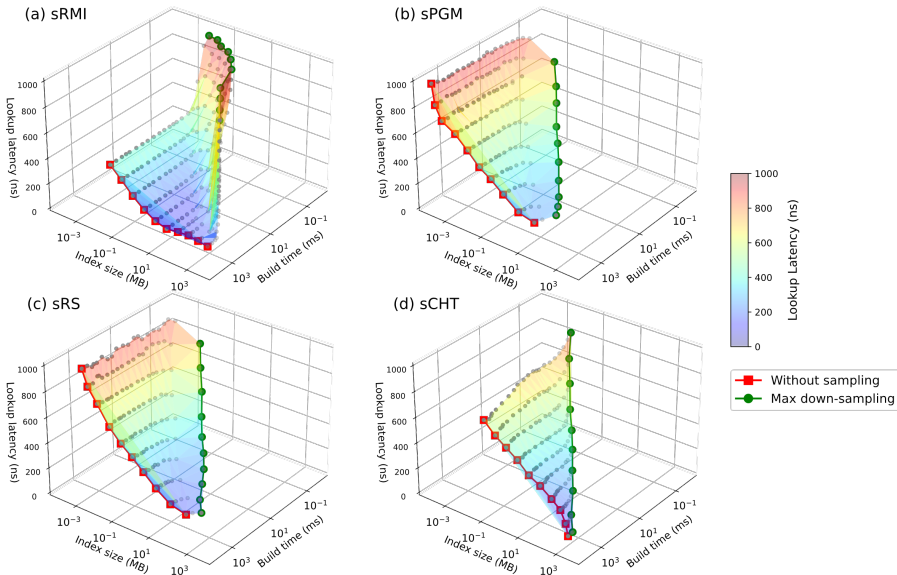


Fig. 11. Design space of indexing schemes on *history* dataset.

5.3 Expanding Design Space of (Learned) Indexes

In this section, we evaluate the indexes with extensive parameters space, and present the performance in 3D space. As an example, we present the performance from the *history* dataset. X, Y, and Z axes present the performance numbers in index size, index-building time, and lookup latency, respectively. In the figure, the red line presents the performance without sampling. Regardless of the indexes, we can see the trade-off between the index size and lookup latency.

From the red line, we apply sampling, and the gray dots present the points according to the sampling interval. We increase the sampling interval from one to 2^{20} . The green line presents the performance points when we have the maximum sampling interval. In the graph, we can observe that sampling expands the design space of learned indexes from a red line on a 2D (index size-lookup time) plane to a 3D curved space, incorporating the build time as a major performance axis.

In Figure 11, sRMI can set the sampling interval independently from the number of segments, which determines the index size. In contrast, error-bounded indexes like sPGM, sRS, and sCHT have maximum intervals determined by the error bound. When the index size is large, the error bound is small, resulting in a relatively narrow design space. Conversely, when the index has a small size, the error bound is increased, leading to a relatively broad design space.

In the figure, sRMI seems to have a large design space because we can apply aggressive sampling even with large index sizes. However, it doesn't significantly reduce build time, and it leads to a substantial increase in lookup latency.

5.4 Pareto Analysis

In this section, we comparatively present the performance of learned indexes with sampling. Figure 12 and 13 shows the Pareto-optimal frontiers of all indexes regarding the build time and lookup latency.

To find the Pareto-optimal configurations, we extensively explored the performance space of indexes over 4,100+ configurations (20+ configurations for tree-based indexes, 400+ for sRMI and sRT, and 1,100+ for sPGM, sRS, and sCHT) on each of the six datasets. For better readability, we regroup the configurations based on the build time. Then, we find the Pareto-optimal points, where there is no shorter build time and lower lookup latency is presented than the current configuration.

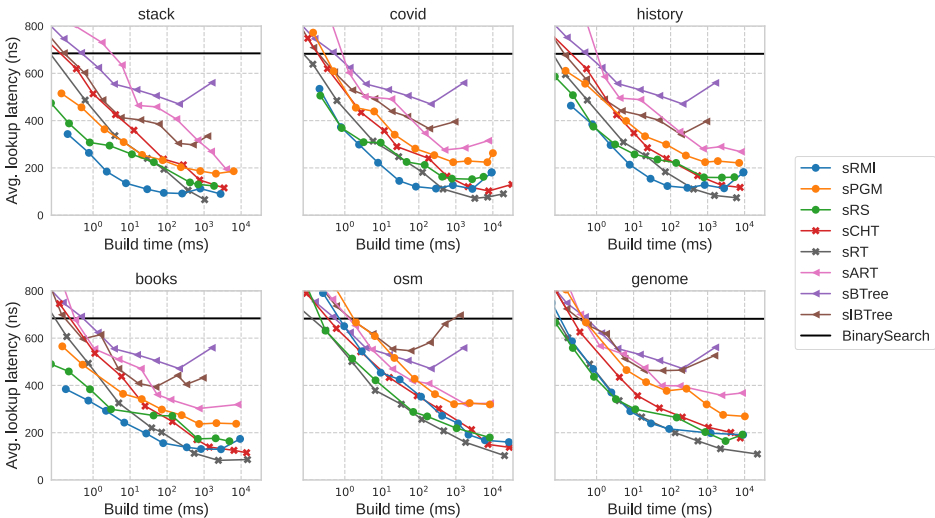


Fig. 12. Average latency by the index build time.

5.4.1 Average Latency.

Figure 12 shows the trade-off between build time and lookup latency. In the figure, across all datasets and build times, learned indexes and histogram indexes outperform tree-based indexes, presenting lower lookup latency and build time. In datasets with low or moderate hardness (*stack*, *covid*, *history*, and *books*), learned indexes are **Pareto-optimal**, with sRMI displaying the lowest latency in the figure: up to 255 ms in *stack*, 290 ms in *covid*, 283 ms in *history*, and 339 ms in *books*. However, in datasets with high hardness (*osm* and *genome*), the performance of learned indexes is similar to that of histogram indexes; sRS and sRMI show comparable lookup latencies with sRT, reaching up to 70 ms in *osm* and 29 ms in *genome*, respectively. In all datasets, sRT presents the lowest latency, particularly when a long build time is allowed.

Learned indexes, which use linear models, are sensitive to the distribution of datasets, resulting in significant performance variability. In contrast, histogram indexes employ constant models, leading to less variability across datasets. To achieve under 200 ns lookup latency, sRMI requires at least 2.3 ms build time in *stack*, 1.2 seconds in *genome*. In contrast, sRT needs at least 81 ms build time in *stack*, 133 ms in *genome*. In easy datasets, learned indexes efficiently learn key distribution with a small amount of training data. On the other hand, if the dataset is difficult, both learned and histogram indexes struggle to effectively learn the key distribution. In such cases, the drawback of sRT's constant model is mitigated, and the advantage of returning a correction range with a single radix operation becomes prominent.

Among the learned indexes, sRMI exhibits the best lookup performance across datasets, excluding *osm*. sRS outperforms sRMI only in *osm* and consistently surpasses sPGM across all datasets and build times. sCHT exhibits lower performance compared to sRT across all datasets and build times, having more layers and a relatively complex build process.

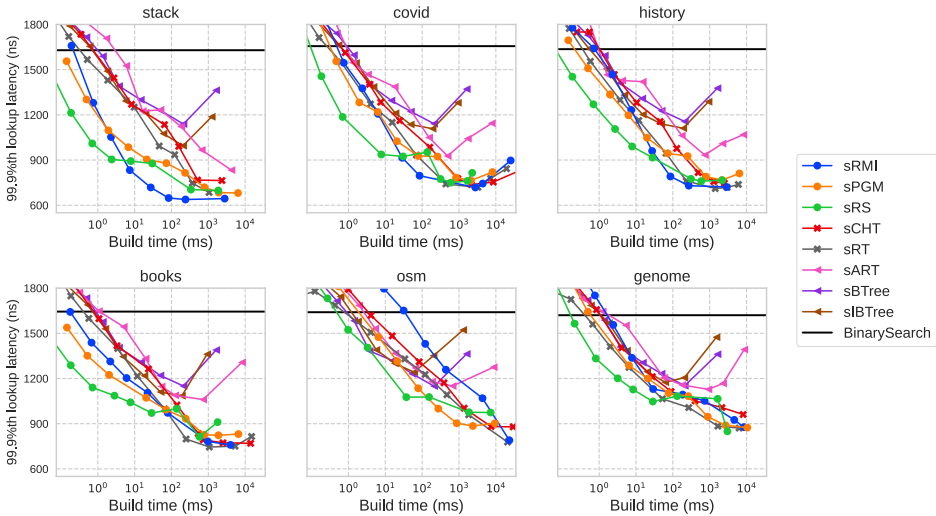


Fig. 13. 99.9th percentile tail latency by the index build time.

5.4.2 99.9th Percentile Tail Latency.

Figure 13 shows the trade-off between build time and 99.9 percentile tail lookup latency (tail latency). Across all build time for every dataset, learned indexes and histogram indexes also outperform tree-based indexes in tail latency. Learned indexes are **Pareto-optimal** as they exhibit the lowest tail latency across all build times in *stack*, up to 315 ms in *covid*, 271 ms in *history*, 78 ms in *books*, 2.3 s in *osm*, and 28 ms in *genome*. From the mentioned point onward, except *stack*, sRT shows

comparable performance in *covid*, *history*, and *genome* and slightly better performance in *books* and *genome*.

Interestingly, sRS and sPGM outperform sRMI in terms of tail latency up to 7 ms in *stack*, 28 ms in *covid*, 28 ms in *history*, 78 ms in *books*, and 9.3 s in *osm*, across all build times in *genome*. For relatively easy datasets, sRS and sPGM show lower latency than sRMI only at shorter build times, but for extremely challenging datasets, they consistently outperform sRMI in tail latency across almost all ranges. This underscores the effectiveness of the Sample EB-PLA training method applied to error-bounded learned indexes in terms of tail latency. Additionally note that although sCHT offers a consistent error bound, its complex build does not yield better tail latency than sRT in terms of build time.

In previous studies, the longer build time of learned indexes compared to traditional and histogram indexes was highlighted as a limitation and prioritized as future work [8, 14, 18, 19, 21, 33, 34, 39]. To the best of our knowledge, our benchmark results are the first to show that learned indexes can be build-efficient in terms of both average and tail latency through sampling, especially in shorter build times.

6 DISCUSSION AND FUTURE WORK

To reduce the model training overhead in learned indexes, approaches include not only reducing the number of elements through sampling but also decreasing the per-element overhead, which is orthogonal to sampling. This can be achieved by leveraging hardware parallelism, such as SIMD and GPU, and using models with a lower per-element overhead [19].

The EB-PLA model is not only used for read-only indexes but also for various updatable learned indexes. In this paper, we focus on the analysis of the impact of sampling on the building of read-only learned indexes; however, we believe that Sample EB-PLA can also efficiently mitigate the model training issues, such as initial building and model re-training, in updatable learned indexes [14, 15, 32].

The sampling interval is one of the important parameters in sampling. In this paper, we focus on the fixed sampling interval for simplicity. Intelligent sampling schemes such as dynamic frequency optimization could further enhance the sampling efficiency. Also, other sampling methods like random and stratified sampling could be considered.

7 CONCLUSIONS

In this paper, we propose a new innovative sampled learning scheme for diverse kinds of learned indexes, and present a new benchmark called BASIL for sampled learning. Up to our knowledge, our Sample EB-PLA and Sample EB-Histogram are the first approaches that successfully apply sampling to error-bounded learned indexes and histogram indexes.

Based on the analysis of the error-bound property and internal segmentation of sRMI, sPGM, sRS, and sCHT, we present a detailed trade-off analysis for the index-building time, index size, lookup latency, internal structure, accuracy, and micro-architecture. With sampling, the index-building time, which is one of the most practical challenges in learned indexes, can be greatly improved in six real-world datasets. We also reveal that sampling allows the build time of learned indexes to form a trade-off with lookup latency and index size, thereby broadening the design space.

Exploring the design space of (learned) indexes over 4,100 configurations with each of the six real-world datasets, Pareto analysis shows that learned indexes can be built more efficiently than traditional indexes through sampling. Besides, Pareto analysis of tail latency highlights the prowess of the proposed error-bounded sample learning algorithm for time-constrained query processing. We believe that our benchmark will make a valuable contribution to the activation and advancement of future learned index studies.

ACKNOWLEDGMENTS

This research was supported by three grants funded by the Ministry of Science and ICT (MSIT), including: (SW StarLab) Development of Novel Key-Value DB for Unstructured Bigdata Research Program (No. 2021-0-01475) and Development of Collection and Integrated Analysis Methods of Automotive Inter/Intra System Artifacts through Construction of Event-based Experimental System Research Program (No. 2022-0-01022), both through the Institute of Information & Communications Technology Planning & Evaluation (IITP); The Basic Science Research Program (No. 2022R1A2C1006050) through the National Research Foundation of Korea (NRF).

REFERENCES

- [1] 2007. STX B+-Tree. Retrieved October 14, 2023 from <https://panthema.net/2007/stx-btree/>
- [2] 2008. CassandraDB. Retrieved October 14, 2023 from <https://github.com/apache/cassandra>
- [3] 2011. C++ Reference. Retrieved October 14, 2023 from https://en.cppreference.com/w/cpp/algorithm/lower_bound
- [4] 2011. LevelDB. Retrieved October 14, 2023 from <https://github.com/google/leveldb>
- [5] 2012. RocksDB. Retrieved October 14, 2023 from <https://github.com/facebook/rocksdb>
- [6] 2019. PGM-Index. Retrieved October 14, 2023 from <https://github.com/gvinciguerra/PGM-index>
- [7] Hussam Abu-Libdeh, Deniz Altınbüken, Alex Beutel, Ed H. Chi, Lyric Doshi, Tim Kraska, Xiaozhou, Li, Andy Ly, and Christopher Olston. 2020. Learned Indexes for a Google-scale Disk-based Database. <http://arxiv.org/abs/2012.12501> [cs].
- [8] Andrew Crotty. 2021. Hist-Tree: Those Who Ignore It Are Doomed to Learn.. In CIDR.
- [9] Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnathan Alagappan, Brian Kroth, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2020. From {WiscKey} to Bourbon: A Learned Index for {Log-Structured} Merge Trees. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 155–171.
- [10] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, David Lomet, and Tim Kraska. 2020. ALEX: An Updatable Adaptive Learned Index. In Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. ACM, Portland OR USA, 969–984. <https://doi.org/10.1145/3318464.3389711>
- [11] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. 2020. Tsunami: a learned multi-dimensional index for correlated data and skewed workloads. Proceedings of the VLDB Endowment 14, 2 (Oct. 2020), 74–86. <https://doi.org/10.14778/3425879.3425880>
- [12] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. 2020. Why are learned indexes so effective?. In International Conference on Machine Learning. PMLR, 3123–3132.
- [13] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. 2021. On the performance of learned data structures. Theoretical Computer Science 871 (June 2021), 107–120. <https://doi.org/10.1016/j.tcs.2021.04.015>
- [14] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. Proceedings of the VLDB Endowment 13, 8 (2020), 1162–1175.
- [15] Alex Galakatos, Michael Markovitch, Carsten Binnig, Rodrigo Fonseca, and Tim Kraska. 2019. FITing-Tree: A Data-aware Index Structure. In Proceedings of the 2019 International Conference on Management of Data. ACM, Amsterdam Netherlands, 1189–1206. <https://doi.org/10.1145/3299869.3319860>
- [16] Jiake Ge, Boyu Shi, Yanfeng Chai, Yuanhui Luo, Yunda Guo, Yinxuan He, and Yunpeng Chai. 2023. Cutting Learned Index into Pieces: An In-depth Inquiry into Updatable Learned Indexes. In 2023 IEEE 39th International Conference on Data Engineering (ICDE). IEEE, 315–327.
- [17] Goetz Graefe. 2006. B-tree indexes, interpolation search, and skew. In Proceedings of the 2nd international workshop on Data management on new hardware. 5–es.
- [18] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2019. SOSD: A Benchmark for Learned Indexes. NeurIPS Workshop on Machine Learning for Systems (2019).
- [19] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2020. RadixSpline: a single-pass learned index. In Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management. ACM, Portland Oregon, 1–5. <https://doi.org/10.1145/3401071.3401659>
- [20] Tim Kraska. 2021. Towards instance-optimized data systems. Proceedings of the VLDB Endowment 14, 12 (July 2021), 3222–3232. <https://doi.org/10.14778/3476311.3476392>
- [21] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In Proceedings of the 2018 International Conference on Management of Data. ACM, Houston TX USA, 489–504. <https://doi.org/10.1145/3183713.3196909>

- [22] Ani Kristo and Tim Kraska. 2023. Parallel External Sorting of ASCII Records Using Learned Models. <http://arxiv.org/abs/2305.05671> arXiv:2305.05671 [cs].
- [23] Ani Kristo, Kapil Vaidya, Ugur Çetintemel, Sanchit Misra, and Tim Kraska. 2020. The Case for a Learned Sorting Algorithm. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 1001–1016. <https://doi.org/10.1145/3318464.3389752>
- [24] Hai Lan, Zhifeng Bao, J. Shane Culpepper, and Renata Borovica-Gajic. 2023. Updatable Learned Indexes Meet Disk-Resident DBMS - From Evaluations to Design Choices. *Proceedings of the ACM on Management of Data* 1, 2 (June 2023), 1–22. <https://doi.org/10.1145/3589284>
- [25] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The adaptive radix tree: ARTful indexing for main-memory databases. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 38–49.
- [26] Pengfei Li, Yu Hua, Jingnan Jia, and Pengfei Zuo. 2021. FINEdex: a fine-grained learned index scheme for scalable and concurrent memory systems. *Proceedings of the VLDB Endowment* 15, 2 (Oct. 2021), 321–334. <https://doi.org/10.14778/3489496.3489512>
- [27] Pengfei Li, Yu Hua, Pengfei Zuo, Zhangyu Chen, and Jiajie Sheng. 2023. {ROLEX}: A Scalable {RDMA-oriented} Learned {Key-Value} Store for Disaggregated Memory Systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 99–114.
- [28] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. 2020. LISA: A Learned Index Structure for Spatial Data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 2119–2133. <https://doi.org/10.1145/3318464.3389703>
- [29] Yaliang Li, Daoyuan Chen, Bolin Ding, Kai Zeng, and Jingren Zhou. 2021. A Pluggable Learned Index Method via Sampling and Gap Insertion. <http://arxiv.org/abs/2101.00808> arXiv:2101.00808 [cs].
- [30] Baotong Lu, Jialin Ding, Eric Lo, Umar Farooq Minhas, and Tianzheng Wang. 2021. APEX: a high-performance learned index on persistent memory. *Proceedings of the VLDB Endowment* 15, 3 (2021), 597–610.
- [31] Kai Lu, Nannan Zhao, Jiguang Wan, Changhong Fei, Wei Zhao, and Tongliang Deng. 2022. TridentKV: A Read-Optimized LSM-Tree Based KV Store via Adaptive Indexing and Space-Efficient Partitioning. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (Aug. 2022), 1953–1966. <https://doi.org/10.1109/TPDS.2021.3118599>
- [32] Chaohong Ma, Xiaohui Yu, Yifan Li, Xiaofeng Meng, and Aishan Maolinyazi. 2022. Film: A fully learned index for larger-than-memory databases. *Proceedings of the VLDB Endowment* 16, 3 (2022), 561–573.
- [33] Marcel Maltry and Jens Dittrich. 2022. A critical analysis of recursive model indexes. *Proceedings of the VLDB Endowment* 15, 5 (Jan. 2022), 1079–1091. <https://doi.org/10.14778/3510397.3510405>
- [34] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2020. Benchmarking learned indexes. *Proceedings of the VLDB Endowment* 14, 1 (Sept. 2020), 1–13. <https://doi.org/10.14778/3421424.3421425>
- [35] Ryan Marcus, Emily Zhang, and Tim Kraska. 2020. CDFShop: Exploring and Optimizing Learned Index Structures. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 2789–2792. <https://doi.org/10.1145/3318464.3384706>
- [36] Mayank Mishra and Rekha Singhal. 2021. RUSLI: Real-time Updatable Spline Learned Index. In *Fourth Workshop in Exploiting AI Techniques for Data Management*. ACM, Virtual Event China, 1–8. <https://doi.org/10.1145/3464509.3464886>
- [37] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. 2020. Learning Multi-Dimensional Indexes. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 985–1000. <https://doi.org/10.1145/3318464.3380579>
- [38] Thomas Neumann and Sebastian Michel. 2008. Smooth Interpolating Histograms with Error Guarantees. In *Sharing Data, Information and Knowledge*, Alex Gray, Keith Jeffery, and Jianhua Shao (Eds.). Vol. 5071. Springer Berlin Heidelberg, Berlin, Heidelberg, 126–138. https://doi.org/10.1007/978-3-540-70504-8_12 ISSN: 0302-9743, 1611-3349 Series Title: Lecture Notes in Computer Science.
- [39] Mihail Stoian, Andreas Kipf, Ryan Marcus, and Tim Kraska. 2021. Towards Practical Learned Indexing. <http://arxiv.org/abs/2108.05117> arXiv:2108.05117 [cs].
- [40] Jinghan Sun, Shaobo Li, Yunxin Sun, Chao Sun, Dejan Vucinic, and Jian Huang. 2023. LeaFTL: A Learning-Based Flash Translation Layer for Solid-State Drives. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, Vancouver BC Canada, 442–456. <https://doi.org/10.1145/3575693.3575744>
- [41] Zhaoan Sun, Xuanhe Zhou, and Guoliang Li. 2023. Learned Index: A Comprehensive Experimental Evaluation. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1992–2004.
- [42] Chuzhe Tang, Youyun Wang, Zhiyuan Dong, Gansen Hu, Zhaoguo Wang, Minjie Wang, and Haibo Chen. 2020. XIndex: a scalable learned index for multicore data storage. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, San Diego California, 308–320. <https://doi.org/10.1145/3332466.3374547>

- [43] Xingda Wei, Rong Chen, Haibo Chen, and Binyu Zang. 2021. XStore : Fast RDMA-Based Ordered Key-Value Store Using Remote Learned Cache. *ACM Transactions on Storage* 17, 3 (Aug. 2021), 1–32. <https://doi.org/10.1145/3468520>
- [44] Chaichon Wongkham, Baotong Lu, Chris Liu, Zhicong Zhong, Eric Lo, and Tianzheng Wang. 2022. Are updatable learned indexes ready? *Proceedings of the VLDB Endowment* 15, 11 (July 2022), 3004–3017. <https://doi.org/10.14778/3551793.3551848>
- [45] Jiacheng Wu, Yong Zhang, Shimin Chen, Jin Wang, Yu Chen, and Chunxiao Xing. 2021. Updatable learned index with precise positions. *Proceedings of the VLDB Endowment* 14, 8 (April 2021), 1276–1288. <https://doi.org/10.14778/3457390.3457393>
- [46] Sai Wu, Zhifei Pang, Gang Chen, Yunjun Gao, Cenjong Zhao, and Shili Xiang. 2021. NEIST: A Neural-Enhanced Index for Spatio-Temporal Queries. *IEEE Transactions on Knowledge and Data Engineering* 33, 4 (April 2021), 1659–1673. <https://doi.org/10.1109/TKDE.2019.2945947>
- [47] Qing Xie, Chaoyi Pang, Xiaofang Zhou, Xiangliang Zhang, and Ke Deng. 2014. Maximum error-bounded Piecewise Linear Representation for online stream approximation. *The VLDB Journal* 23, 6 (Dec. 2014), 915–937. <https://doi.org/10.1007/s00778-014-0355-0>
- [48] Geoffrey X. Yu, Markos Markakis, Andreas Kipf, Per-Åke Larson, Umar Farooq Minhas, and Tim Kraska. 2022. TreeLine: an update-in-place key-value store for modern storage. *Proceedings of the VLDB Endowment* 16, 1 (Sept. 2022), 99–112. <https://doi.org/10.14778/3561261.3561270>
- [49] Sepanta Zeighami and Cyrus Shahabi. 2023. On distribution dependent sub-logarithmic query time of learned indexing. In *International Conference on Machine Learning*. PMLR, 40669–40680.

Received October 2023; accepted January 2024